

what's under the switch?

digitalisierung und programmierung · prof. dr. nicolas meseth

- 1 what you don't need to know
- 2 your own stack
- 3 swap the bottom
- 4 what it costs

part 1

what you don't need to know

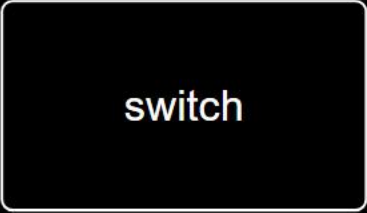
what happens
between your
finger and the
light?



image: ai-generated (gpt-image-2)

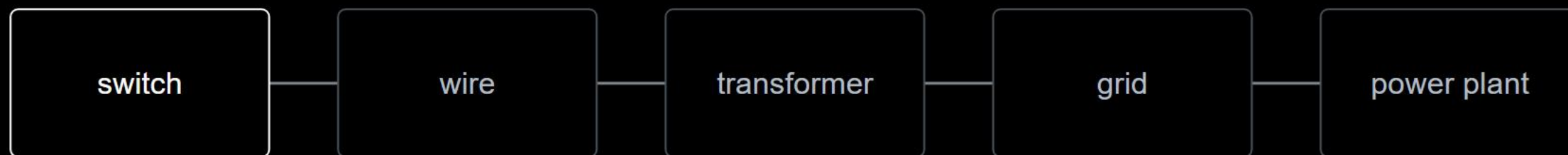
what you don't need to know

behind the switch



switch

behind the switch



behind the switch



you never think about any of this

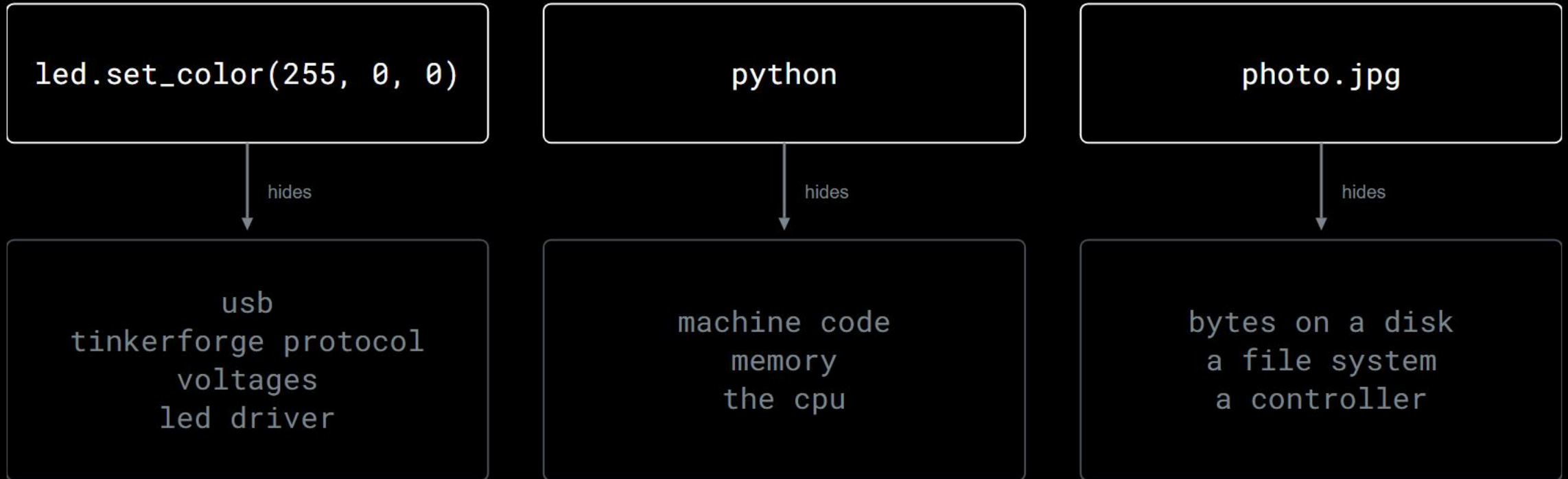
the switch is the whole contract: up, down.

so here is the word for it

**an abstraction hides how something works
and shows what you can do with it.**

the part it shows you is called the interface.

three you used today without noticing



part 2

your own stack

from a photo to light, one layer at a time

sending

meaning



your photo



from a photo to light, one layer at a time

sending

meaning

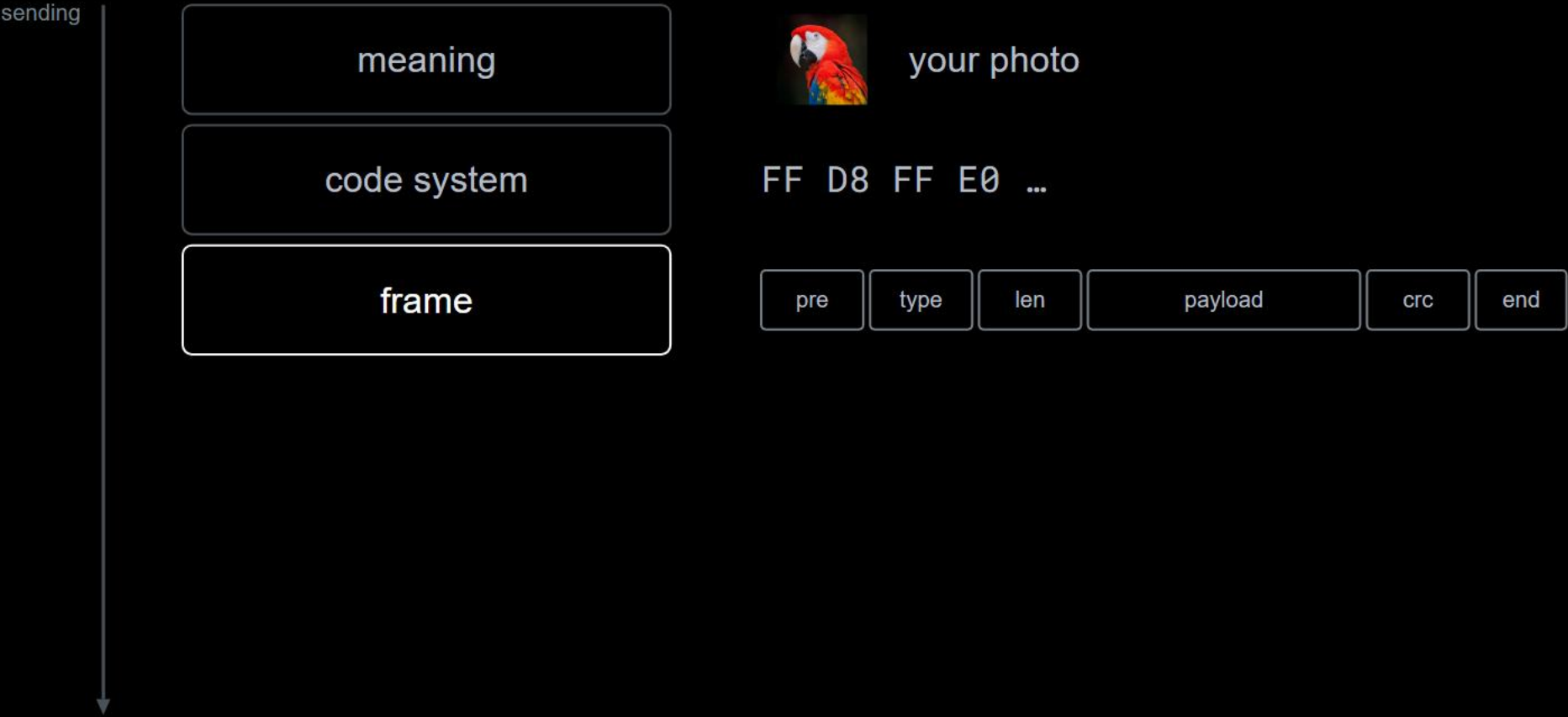
code system



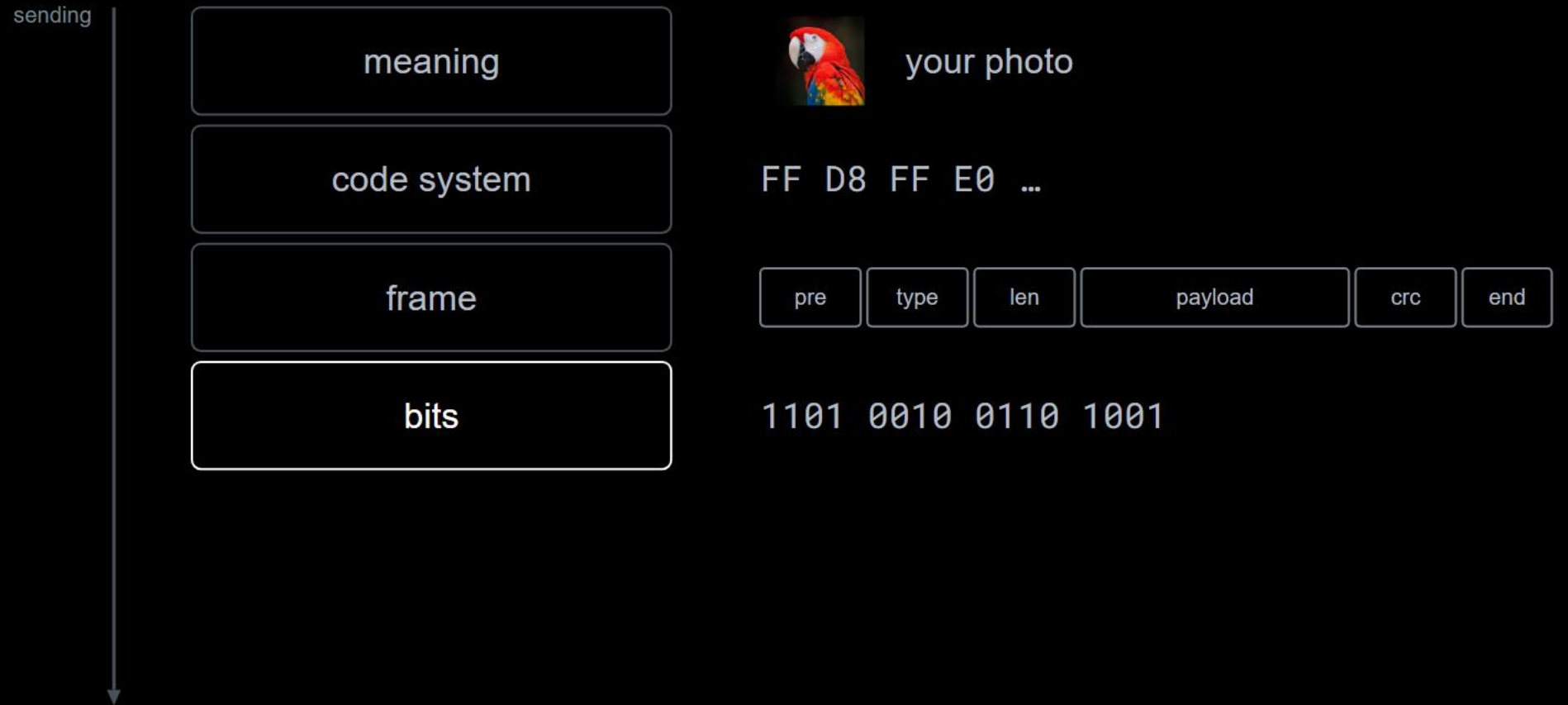
your photo

FF D8 FF E0 ...

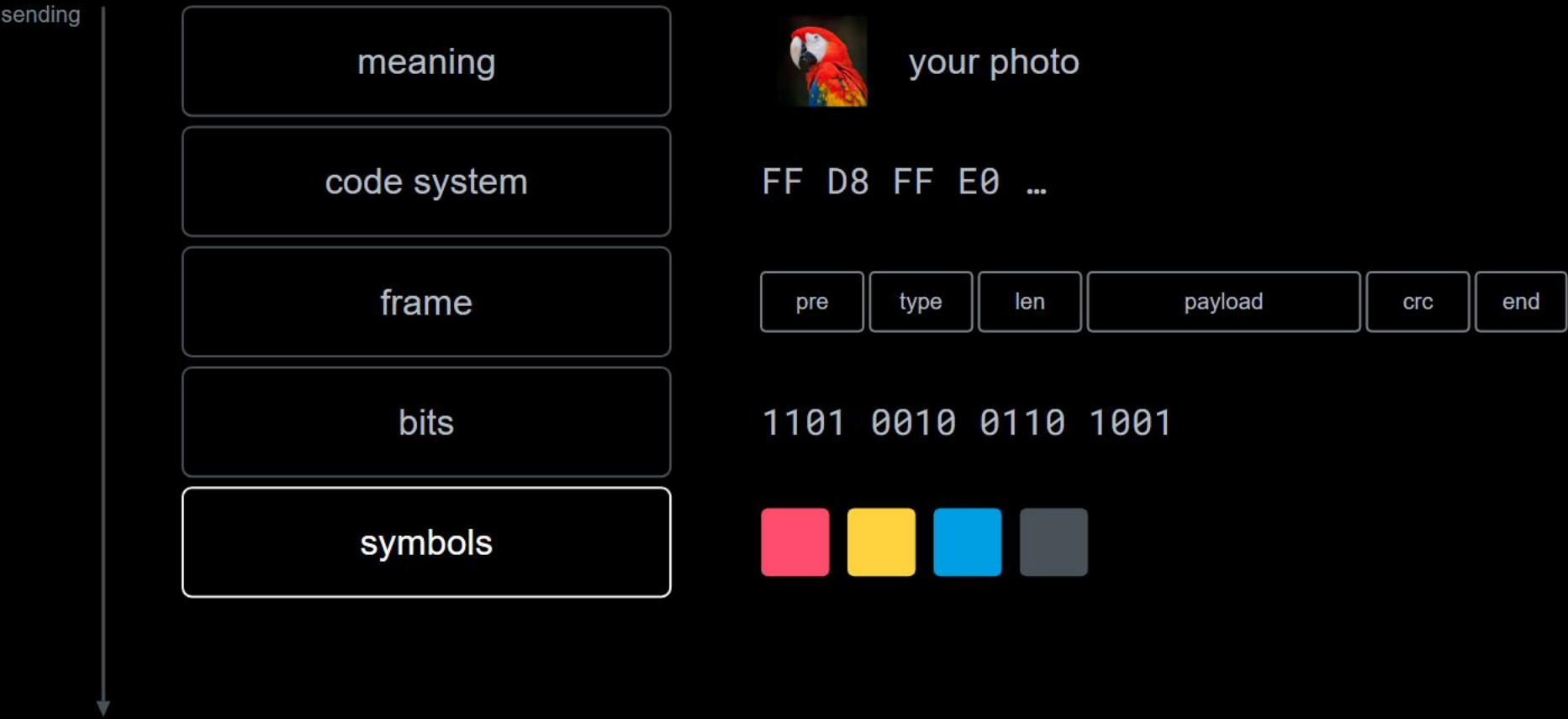
from a photo to light, one layer at a time



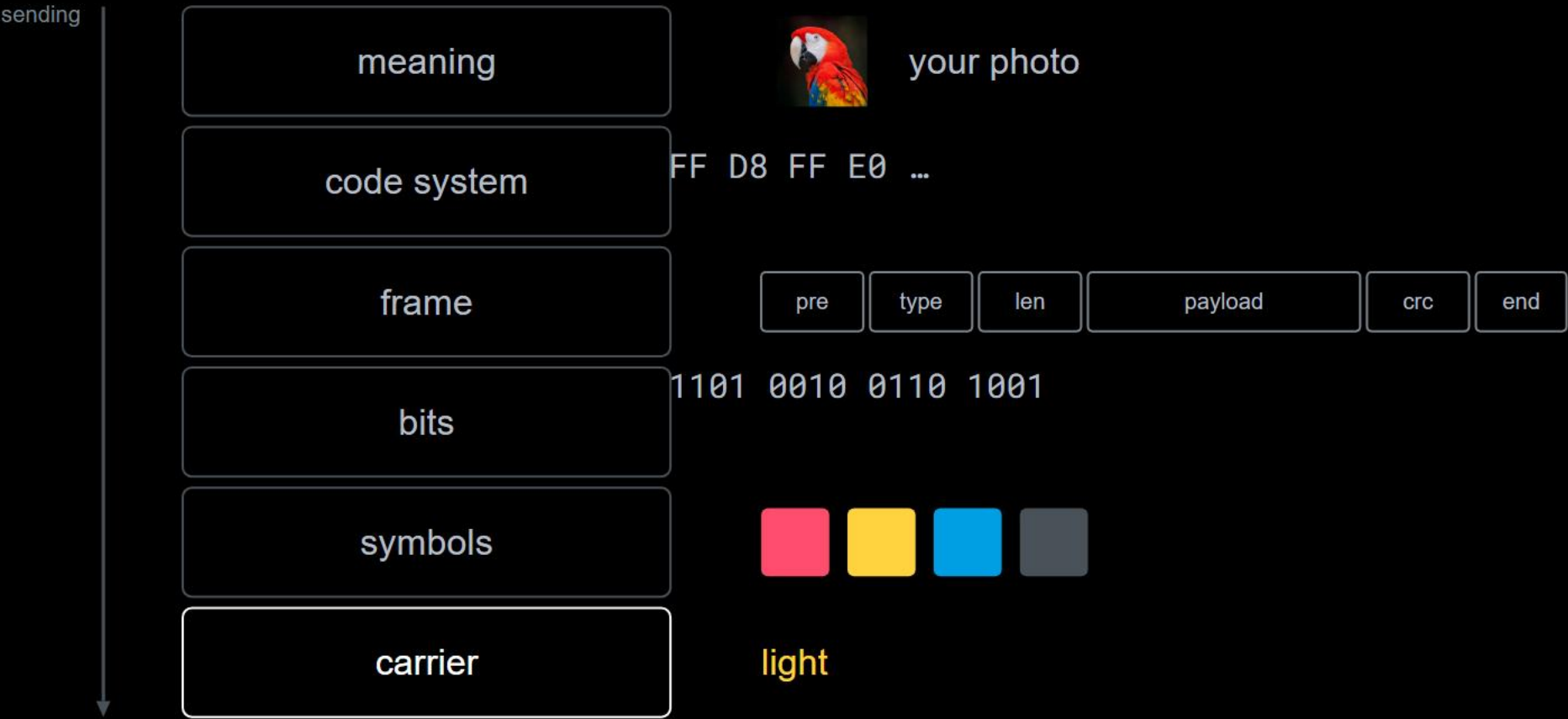
from a photo to light, one layer at a time



from a photo to light, one layer at a time



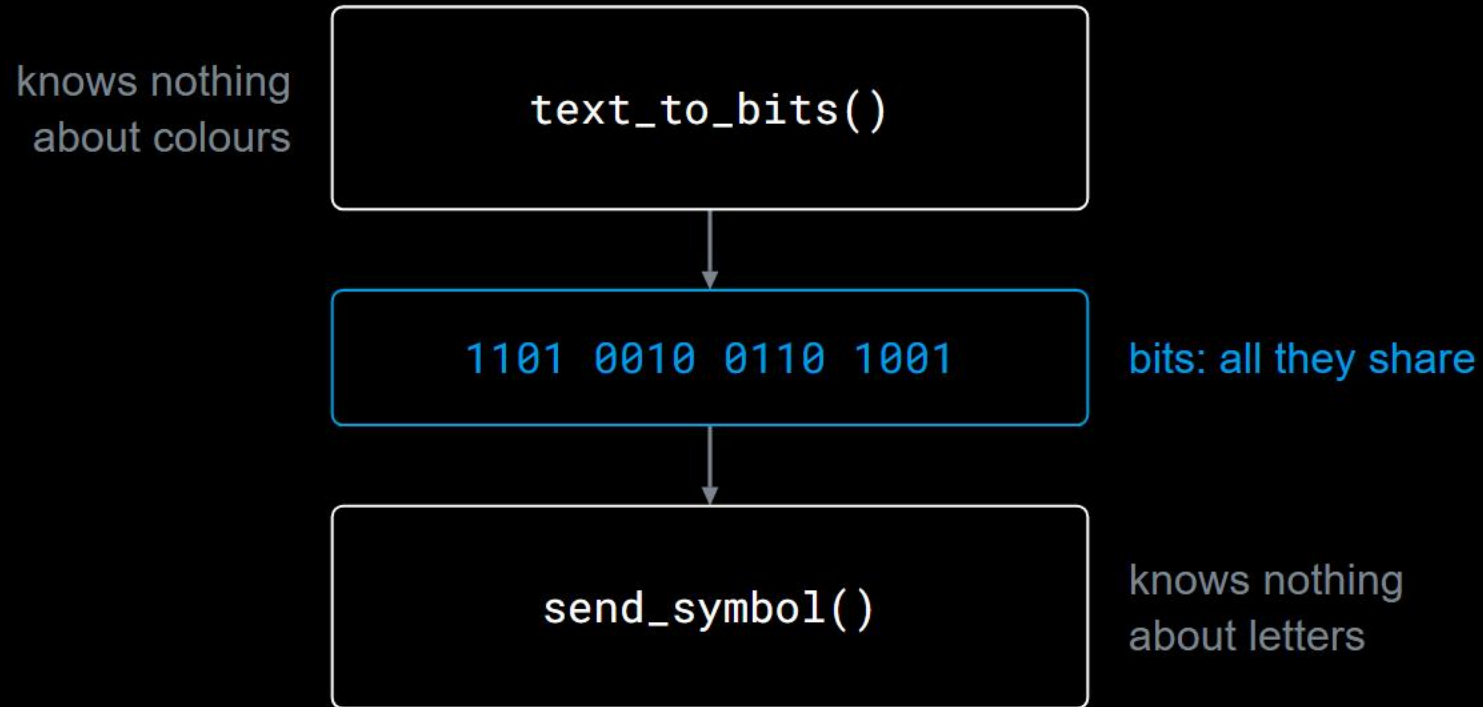
from a photo to light, one layer at a time



each layer talks only to the one
directly below it.

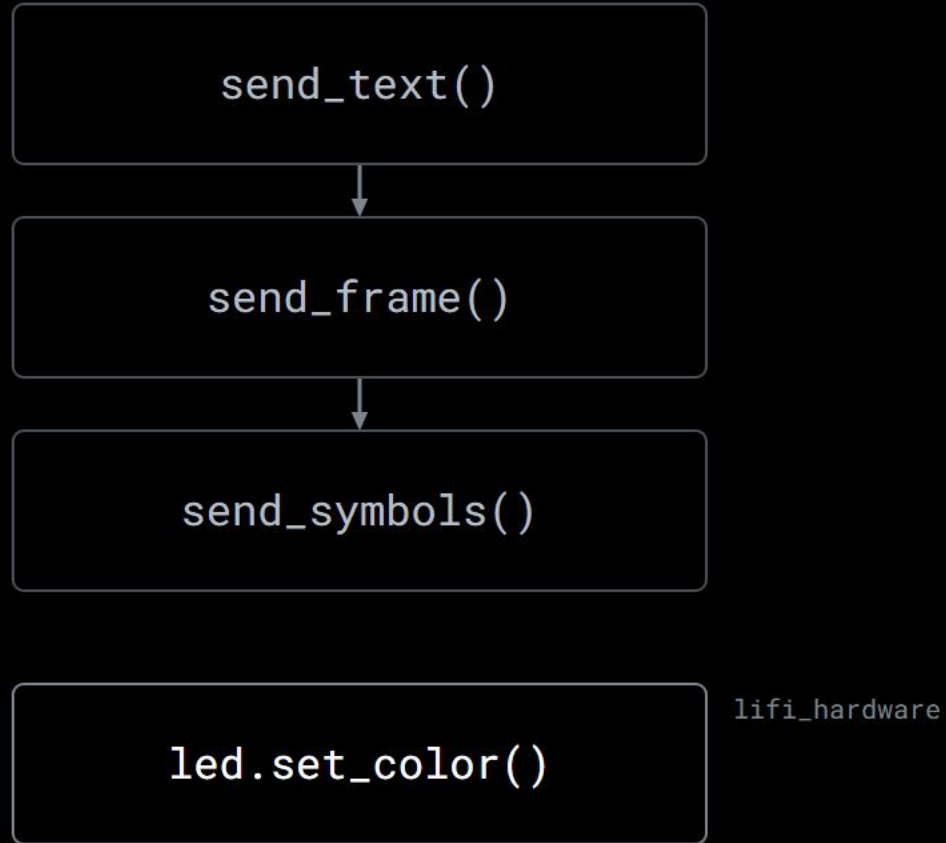
that single rule is what makes everything else possible.

what that buys you

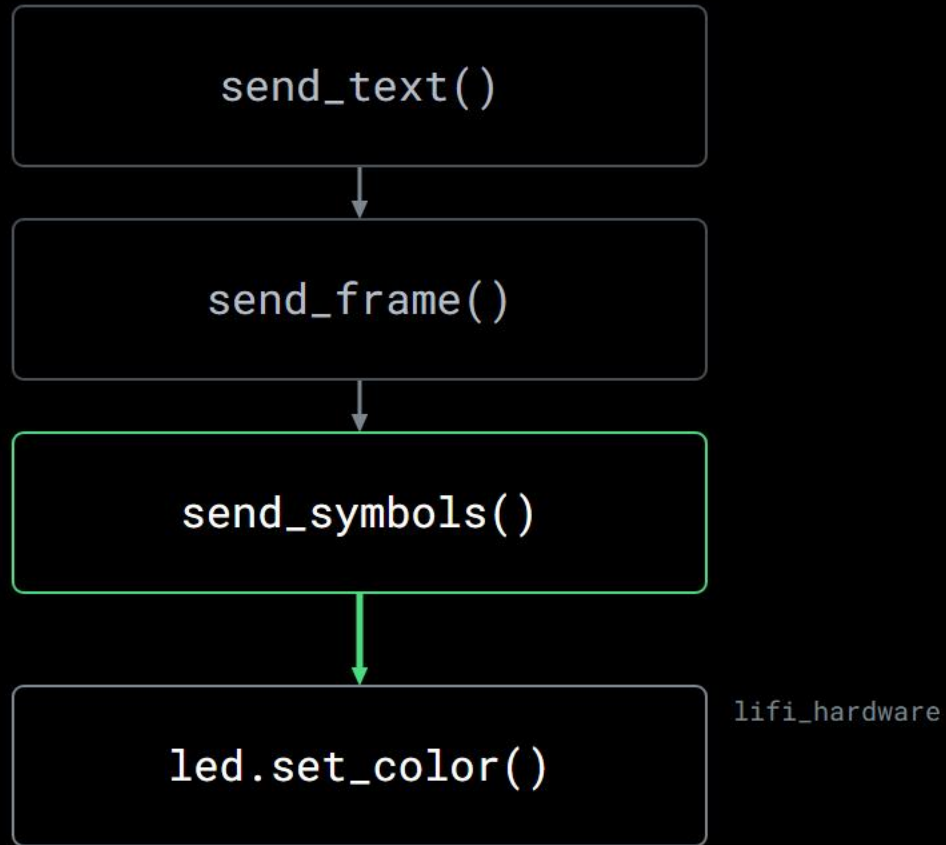


not tidiness. this is why one can change without the other.

which of these may talk to the led?



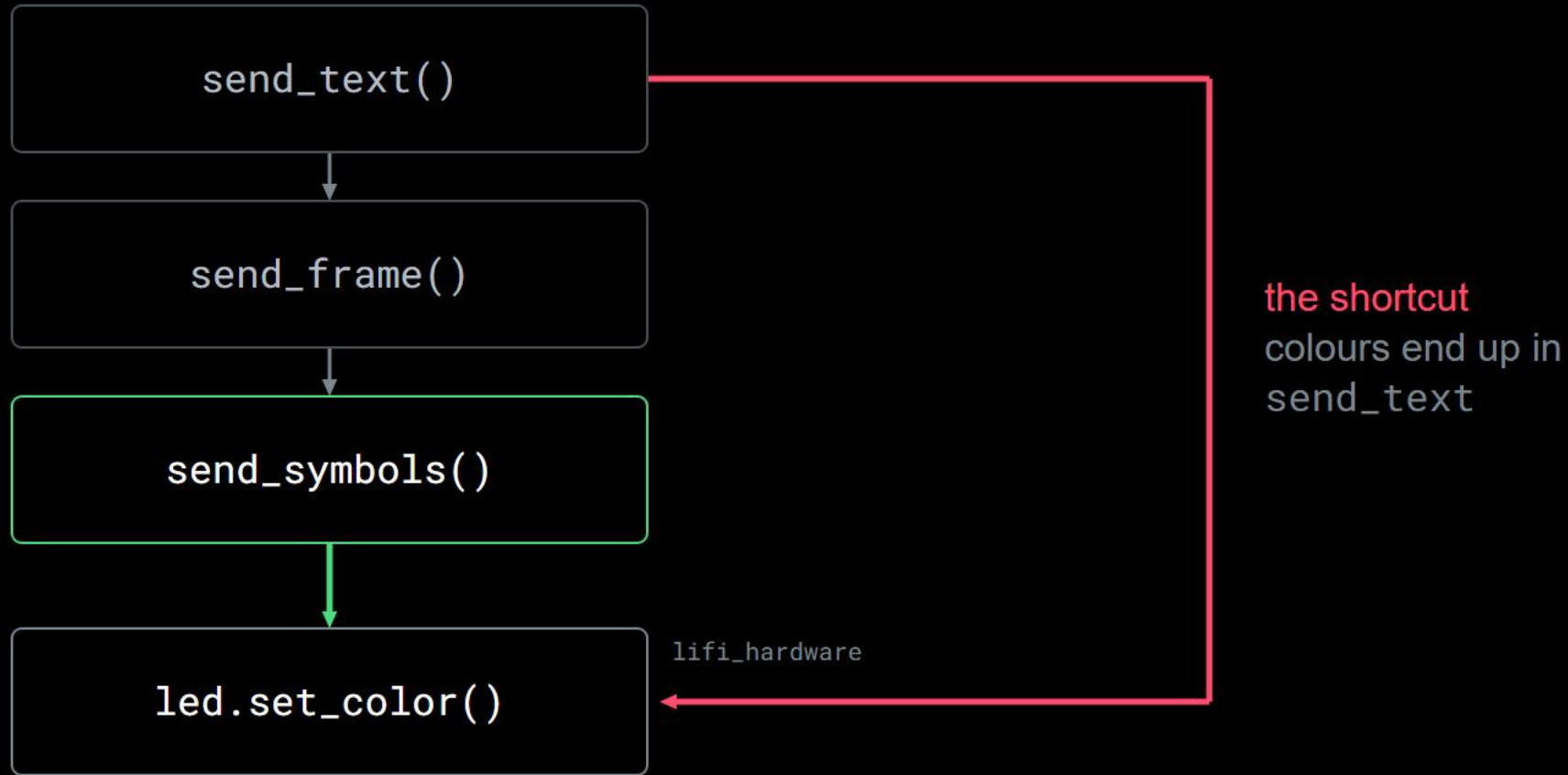
which of these may talk to the led?



allowed

only this layer knows which colour a symbol is

which of these may talk to the led?



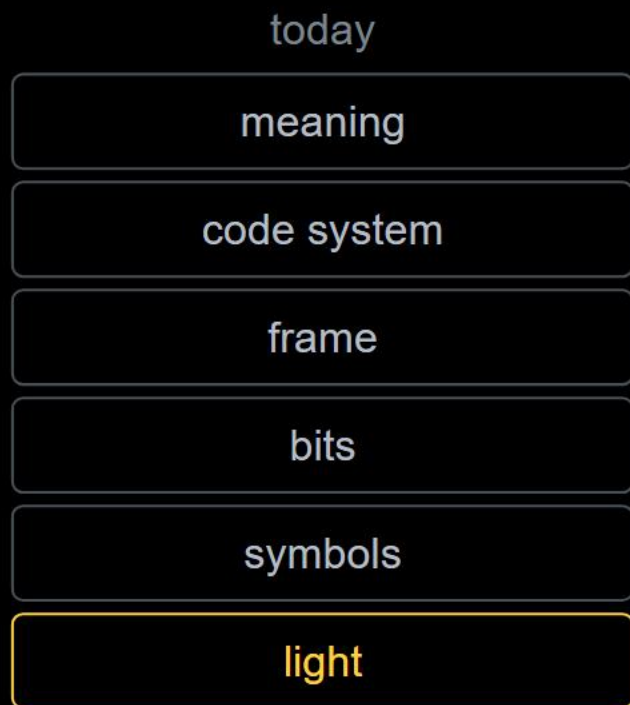
allowed

only this layer knows which colour a symbol is

part 3

swap the bottom

same stack, different bottom



same stack, different bottom



five layers unchanged. one exchanged.

the same bits, three carriers

**information needs a carrier.
it is not the carrier.**

light: a colour on the led, read by a colour sensor

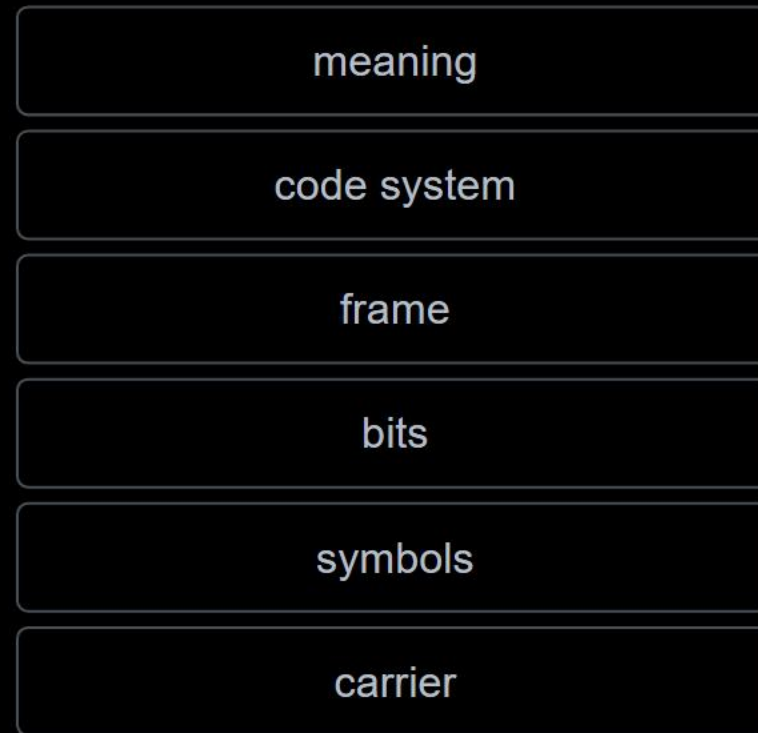
a cable: a voltage, high or low

sound: a tone from the speaker, heard by a microphone

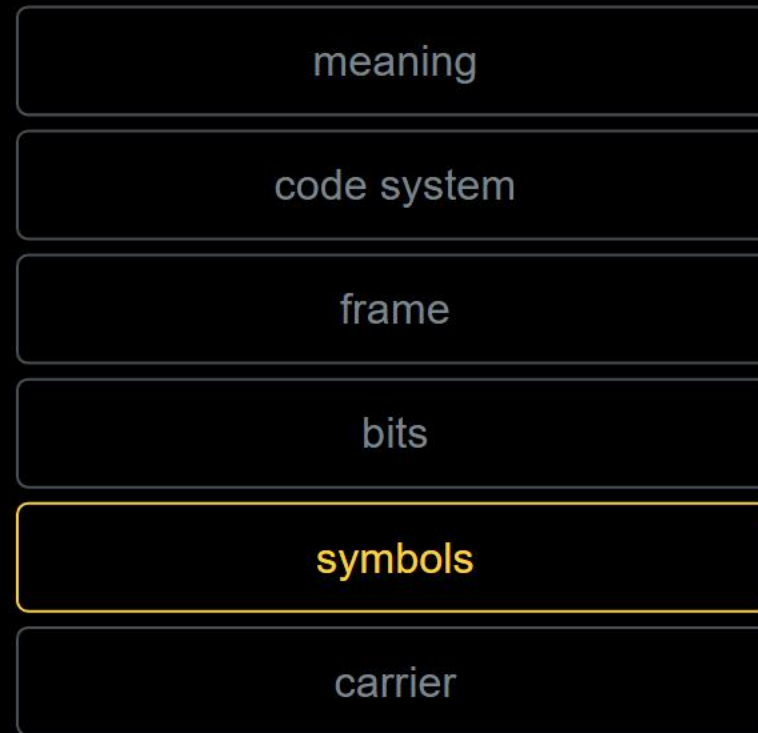
the bit stream is the same in all three. only the bottom layer knows which one it rides on.

what you send is the bits. light is just what carries them today.

you extend your alphabet from four colours to eight



you extend your alphabet from four colours to eight



colour table
calibration
bits ↔ symbols

every other layer: unchanged

a frame is defined in bits, not in colours

and it says nothing about time.

time belongs to the path, not to the message. that is why sound works.

everything here sits on light.

everything here sits on light.

light is the layer you could replace.

part 4

what it costs

layers are not free

- 1 **every layer adds its own data**
the frame costs bytes you cannot use

layers are not free

- 1 **every layer adds its own data**
the frame costs bytes you cannot use
- 2 **every boundary hides a knob**
the one you need is always the hidden one

layers are not free

- 1 **every layer adds its own data**
the frame costs bytes you cannot use
- 2 **every boundary hides a knob**
the one you need is always the hidden one
- 3 **you must learn where the boundaries are**
a wrong guess sends you looking in the wrong place

wrong letters arrive. where do you look first?

meaning · code system · frame · bits

symbols

?

carrier

wrong letters arrive. where do you look first?

meaning · code system · frame · bits

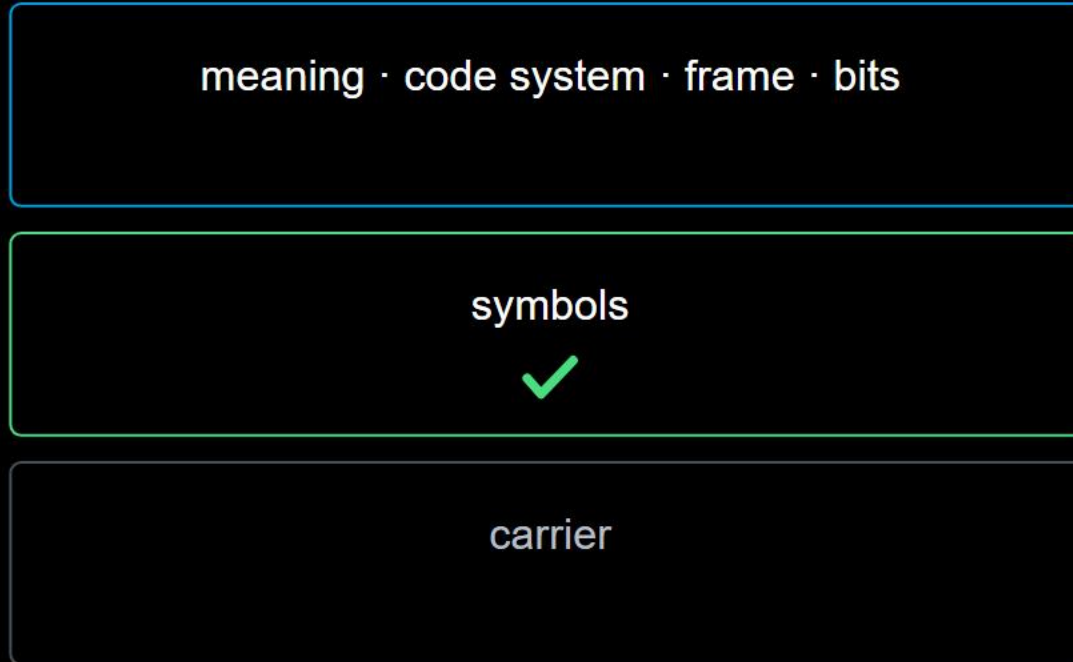
symbols



tested: 100 % of symbols
recognised correctly

carrier

wrong letters arrive. where do you look first?



look here

the fault is above the tested layer

tested: 100 % of symbols
recognised correctly

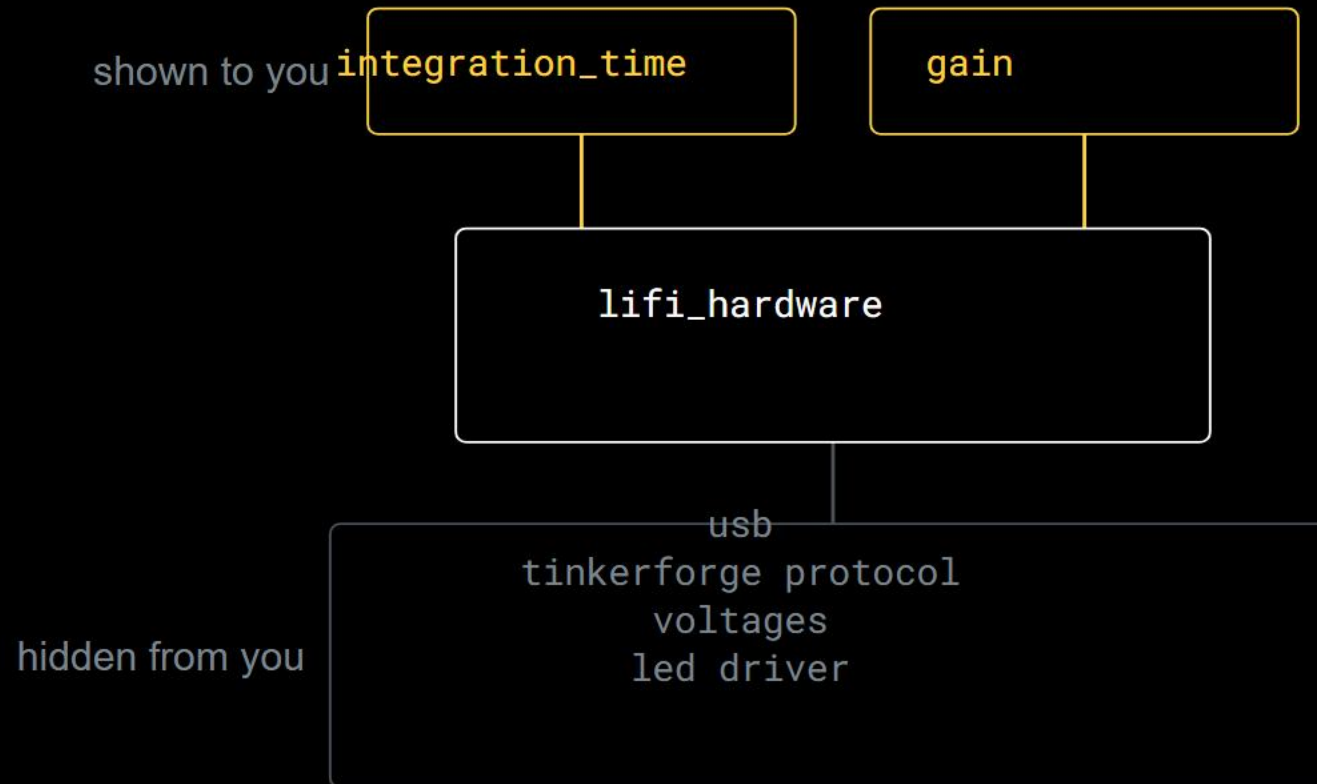
a tested layer is a layer you can stop suspecting.

how to test one layer

feed it a known input, check its output,
with nothing below it.

bits in, bits out. no led, no sensor, no room light.

why doesn't it hide everything?



a good interface hides what you don't need and shows what you must decide.

you know
what's under
the switch.
every layer you
write is one.

